

✓ Lab 08

✓ CÁ NHÂN HÓA THEO MSSV

s	2212453	g	3
d1	2	h	16
d2	2	η	0.002
d3	1		
d4	2		
d5	4		
d6	5		
d7	3		

✓ PHẦN 1. THỰC HÀNH EXCEL THEO BẢNG MẪU

✓ Yêu cầu phải trả lời

1. Nút ẩn đã giúp mô hình biểu diễn điều gì mà mô hình tuyến tính khó làm được?

Nút ẩn, khi được kết hợp với các hàm kích hoạt (như ReLU hay Sigmoid), giúp mô hình biểu diễn được các mối quan hệ phi tuyến tính (non-linear relationships) và những tương tác phức tạp giữa các đặc trưng đầu vào. Các mô hình tuyến tính đơn thuần chỉ có thể tạo ra các đường thẳng hoặc mặt phẳng để phân tách dữ liệu, nên sẽ thất bại với các tập dữ liệu đan xen phức tạp (ví dụ như bài toán XOR). Nút ẩn giúp mạng Nơ-ron "bẻ cong" và chuyển đổi không gian dữ liệu, từ đó ranh giới phân lớp trở nên linh hoạt hơn rất nhiều.

2. Nếu một nút ẩn gần như luôn cho đầu ra 0, điều đó gợi ý gì?

Hiện tượng này thường mang lại hai gợi ý chính: Hiện tượng "Nơ-ron chết" (Dying ReLU): Nếu bạn đang dùng hàm ReLU ($a = \max(0, z)$), khi tổng trọng số đầu vào (z) liên tục bị âm, hàm kích hoạt sẽ ép đầu ra về 0. Lỗi này thường do việc khởi tạo trọng số không tốt hoặc learning rate quá lớn khiến nơ-ron không bao giờ phục hồi và cập nhật được nữa. Sự dư thừa của mạng: Nút ẩn này đang không học được bất kỳ đặc trưng (feature) nào hữu ích để đóng góp vào kết quả dự đoán cuối cùng. Mạng nơ-ron đang có kích thước lớn hơn mức cần thiết cho bài toán, và việc loại bỏ nút ẩn này gần như không làm ảnh hưởng đến độ chính xác của mô hình.

3. Tại sao khi số nút ẩn tăng quá mạnh, mô hình có thể học quá mức (Overfitting)?

Khi số lượng nút ẩn tăng lên quá mạnh, số lượng tham số cần học (các trọng số w và bias b) của mô hình sẽ tăng theo cấp số nhân. Điều này cung cấp cho mô hình một "dung lượng bộ nhớ" (capacity) khổng lồ. Lúc này, thay vì tìm ra những quy luật tổng quát chung của dữ liệu, mô hình lại bắt đầu học thuộc lòng (memorize) từng điểm dữ liệu cụ thể trong tập huấn luyện, bao gồm cả các điểm nhiễu (noise) và các điểm dị biệt (outliers). Hậu quả là mô hình trả về Loss cực thấp trên tập Train nhưng lại dự đoán sai lệch hoàn toàn trên dữ liệu Test thực tế.

✓ Bài tập tự làm

- Tự thêm một nút ẩn thứ ba trên Excel hoặc thử thay đổi hàm kích hoạt của một nút ẩn rồi quan sát output đầu ra đổi ra sao.

Khi thay đổi hàm kích hoạt của 1 nút ẩn thì:

- zout thay đổi do giá trị của nút ẩn 1 truyền vào lúc này tăng từ 0.122 (ReLU) lên khoảng 0.530 (Sigmoid).
- Kéo theo đó, xác suất (probability) sẽ thay đổi (tăng lên).

Cuối cùng, Loss cũng tự động tính lại ra một con số hoàn toàn mới.

=> Việc thay đổi hàm kích hoạt tại một nút ẩn làm thay đổi độ lớn của tín hiệu được truyền đi, tạo ra hiệu ứng dây chuyền làm thay đổi toàn bộ xác suất dự đoán và độ lỗi (Loss) ở đầu ra cuối cùng, dù dữ liệu đầu vào không hề thay đổi.

- Nhận xét việc tăng độ phức tạp của mạng có làm mô hình dễ hiểu hơn hay ngược lại.

Việc tăng độ phức tạp của mạng (như thêm số nút ẩn, thêm lớp ẩn) không làm mô hình dễ hiểu hơn, mà ngược lại, làm nó trở nên khó giải thích hơn rất nhiều (mang đặc tính "hộp đen" - black

box).

Khi mạng nơ-ron có cấu trúc nhỏ gọn (1-2 nút) như trong file Excel vừa thực hành, con người có thể dễ dàng dò theo từng bước tính toán để thấy được tác động của từng trọng số lên kết quả cuối cùng. Tuy nhiên, khi tăng số nút ẩn lên con số hàng chục hay hàng trăm, các ma trận trọng số sẽ đan chéo nhau một cách chằng chịt và các hàm kích hoạt phi tuyến tính sẽ bóp méo dữ liệu liên tục qua nhiều tầng. Lúc này, việc giải thích ý nghĩa cụ thể của một trọng số, hay việc hiểu được một nút ẩn đang đại diện cho đặc trưng thực tế nào của dữ liệu là điều gần như bất khả thi. Mạng nơ-ron đánh đổi đi tính minh bạch và khả năng diễn giải (interpretability) để lấy được sức mạnh tính toán và độ chính xác vượt trội.

✓ Minh chứng

- 01 ảnh toàn màn hình Excel có Formula Bar hiển thị công thức forward pass ở lớp ẩn.

The screenshot shows an Excel spreadsheet with the following data:

STT	Ký hiệu	Giá trị
1	x1	0.2
2	x2	0.2
3	w11	0.25
4	w21	0.22
5	w12	-0.14
6	w22	0.3
7	b1	0.1
8	b2	0.1
9	v1	0.5
10	v2	-0.4
11	b_out	0.2

STT	Ký hiệu	Kết quả	Công thức
1	z1	0.122	$C13 \cdot C15 + C14 \cdot C17 + C19$
2	a1 [Sigmoid]	0.530462226	$1/(1+EXP(-H13))$
3	a1 [ReLU]	0.122	$MAX(0, H13)$
4	z2	0.204	$C13 \cdot C16 + C14 \cdot C18 + C20$
5	a2 [Sigmoid]	0.550823865	$1/(1+EXP(-H16))$
6	a2 [ReLU]	0.204	$MAX(0, H16)$

STT	Ký hiệu	Giá trị
1	x1	0.2
2	x2	0.2
3	w11	0.25
4	w21	0.22
5	w12	-0.14
6	w22	0.3
7	b1	0.1
8	b2	0.1
9	v1	0.5
10	v2	-0.4
11	b_out	0.2

- 01 ảnh toàn màn hình Excel có Formula Bar hiển thị công thức xác suất hoặc loss ở đầu ra.

AutoSave Off 2212453_NgoBaTai_Lab8.xlsx Search Sign in

File Home Insert Draw Page Layout Formulas Data Review View Automate Help

Clipboard Font Alignment Number Styles Cells Editing Add-ins

G31 $=-(G29*LN(G30) + (1-G29)*LN(1-G30))$

BẢNG 1 — ĐẦU VÀO & TRONG SỐ			BẢNG 2 — TÍNH NÚT ẨN: z1, a1, z2, a2			BẢNG 3 — ĐẦU RA			BẢNG 4 — LOSS			BẢNG 5 — SO SÁNH TRỰC GIÁC: Có vs Không có lớp ẩn		
STT	Ký hiệu	Giá trị	STT	Ký hiệu	Kết quả	STT	Ký hiệu	Kết quả	STT	Ký hiệu	Kết quả	STT	Tiêu chí	Kết luận
1	x1	0.2	1	z1	0.122	1	zout	0.1794	1	y (nhân thất)	1	1	Công thức	MLP phức tạp hơn
2	x2	0.2	2	a1 (Sigmoid)	0.530462226	2	probability	0.5447	2	p (xác suất)	0.5447	2	Xác suất đầu ra	Cùng input, khác output
3	w11	0.25	3	a1 (ReLU)	0.122	3	Nhân dự đoán	1	3	Loss (BCE)	0.607520095	3	Số tham số	MLP linh hoạt hơn
4	w21	0.22	4	z2	0.204				4			4	Đường biên phân lớp	MLP tổng quát hơn
5	w12	-0.14	5	a2 (Sigmoid)	0.550823865				5			5	Bài toán XOR	Lớp ẩn là chìa khóa
6	w22	0.3	6	a2 (ReLU)	0.204				6			6	Rủi ro chính	Early Stopping giải quyết
7	b1	0.1												
8	b2	0.1												
9	v1	0.5												
10	v2	-0.4												
11	b_out	0.2												

Phần 1 Sheet1

- 01 ảnh bảng tính tổng thể thể hiện đủ các ô đầu vào, nút ẩn và đầu ra.

BẢNG 1 — ĐẦU VÀO & TRONG SỐ			BẢNG 2 — TÍNH NÚT ẨN: z1, a1, z2, a2			BẢNG 3 — ĐẦU RA			BẢNG 4 — LOSS			BẢNG 5 — SO SÁNH TRỰC GIÁC: Có vs Không có lớp ẩn		
STT	Ký hiệu	Giá trị	STT	Ký hiệu	Kết quả	STT	Ký hiệu	Kết quả	STT	Ký hiệu	Kết quả	STT	Tiêu chí	Kết luận
1	x1	0.2	1	z1	0.122	1	zout	0.1794	1	y (nhân thất)	1	1	Công thức	MLP phức tạp hơn
2	x2	0.2	2	a1 (Sigmoid)	0.53	2	probability	0.5447	2	p (xác suất)	0.5447	2	Xác suất đầu ra	Cùng input, khác output
3	w11	0.25	3	a1 (ReLU)	0.122	3	Nhân dự đoán	1	3	Loss (BCE)	0.6074	3	Số tham số	MLP linh hoạt hơn
4	w21	0.22	4	z2	0.204				4			4	Đường biên phân lớp	MLP tổng quát hơn
5	w12	-0.14	5	a2 (Sigmoid)	0.55				5			5	Bài toán XOR	Lớp ẩn là chìa khóa
6	w22	0.3	6	a2 (ReLU)	0.204				6			6	Rủi ro chính	Early Stopping giải quyết
7	b1	0.1												
8	b2	0.1												
9	v1	0.5												
10	v2	-0.4												
11	b_out	0.2												

- 4–6 dòng giải thích vai trò của lớp ẩn.

Lớp ẩn (hidden layer) đóng vai trò cốt lõi tạo nên sức mạnh của mạng nơ-ron, giúp mô hình biểu diễn và học được các mối quan hệ phi tuyến tính mà các thuật toán tuyến tính thông thường không thể làm được. Nhờ sự kết hợp giữa các tham số (trọng số, bias) và hàm kích hoạt (như ReLU hay Sigmoid), lớp ẩn tự động kết hợp các đặc trưng đầu vào để tạo ra những biểu diễn dữ liệu mới trừu tượng và phức tạp hơn. Quá trình biến đổi này giúp mạng "bề cong" không gian dữ liệu, từ đó vẽ ra được các ranh giới phân lớp uốn lượn để giải quyết các tập dữ liệu đan xen khó (điển hình như bài toán XOR). Tuy nhiên, số lượng nút ẩn cần được thiết lập vừa đủ; nếu quá ít sẽ gây thiếu hụt dung lượng học (underfitting), còn quá nhiều sẽ dễ dẫn đến rủi ro "học thuộc lòng" cả nhiễu (overfitting). Nhìn chung, lớp ẩn chính là bộ xử lý trung tâm, giúp chuyển hóa dữ liệu thô ban đầu thành những đặc trưng mang tính quyết định cao nhất trước khi xuất ra dự đoán cuối cùng.

✓ PHẦN 2. ORANGE DATA MINING TRÊN REAL DATASET

✓ Các yêu cầu phải trả lời

1. Đổi số nút ẩn có làm kết quả cải thiện rõ ràng hay không?

Dựa vào bảng kết quả Test & Score trên Orange, việc thay đổi (cụ thể là tăng) số nút ẩn không làm kết quả cải thiện rõ ràng. Cả hai cấu hình đều đạt chỉ số độ chính xác (CA - Classification Accuracy) và F1-score giống hệt nhau ở mức 0.980. Sự chênh lệch chỉ xuất hiện ở mức độ cực kỳ nhỏ không đáng kể tại chỉ số AUC (0.999 so với 0.998). Điều này chứng tỏ việc nhồi nhét thêm số nút ẩn không mang lại giá trị gia tăng nào cho mô hình ở bài toán này.

2. Nếu kết quả chỉ thay đổi rất ít, điều đó có thể nói gì về độ khó của dữ liệu?

Kết quả gần như không đổi khi tăng độ phức tạp của mô hình cho thấy bộ dữ liệu này có độ khó thấp và tính phân tách tuyến tính rất cao. Các đặc trưng đầu vào của viên kim cương (như trọng lượng carat, kích thước x, y, z) có mối tương quan rất rõ ràng, mạnh mẽ và trực tiếp với nhãn mục tiêu (giá premium). Vì ranh giới phân lớp đơn giản và rõ rệt, mô hình chỉ cần một số lượng ít nút ẩn là đã đủ sức biểu diễn toàn bộ quy luật của dữ liệu; phần nút ẩn dư thừa không có thêm thông tin mới nào để học.

3. Mô hình MLP trong Orange có ưu thế gì so với Logistic Regression ở bài toán này?

Thực tế trên bộ dữ liệu diamonds, MLP không mang lại ưu thế vượt trội về mặt hiệu năng (độ chính xác của cả hai đều xấp xỉ 98%), thậm chí MLP còn tốn thời gian huấn luyện hơn và khó giải thích hơn (hộp đen). Tuy nhiên, xét về mặt đặc tính kỹ thuật, ưu thế của MLP so với Logistic Regression là khả năng tự động nắm bắt các mối quan hệ phi tuyến tính và sự tương tác phức tạp giữa các biến (ví dụ: sự cộng hưởng giữa màu sắc, độ trong suốt và giác cắt để định giá kim cương). Với Logistic Regression, để học được điều này, bạn thường phải tự tạo ra các biến tương tác (feature engineering) bằng tay, trong khi MLP có thể tự động trích xuất các đặc trưng này thông qua các lớp ẩn.

✓ Bài tập tự làm

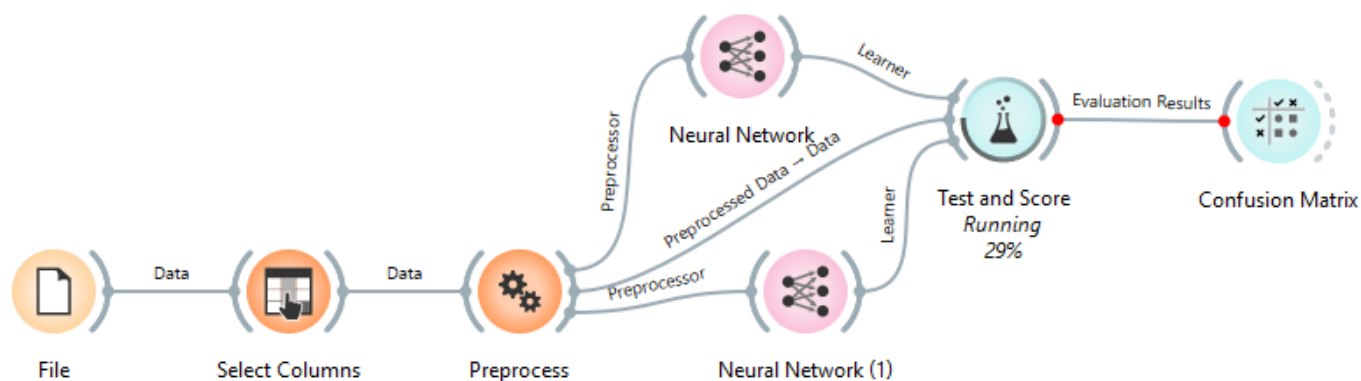
So sánh MLP và mô hình Baseline (Logistic Regression):

Nhìn vào kết quả trên bảng Test & Score, mô hình baseline Logistic Regression đạt độ chính xác (CA) tương đương, thậm chí nhỉnh hơn so với mạng nơ-ron MLP (khoảng 98%). Điều này cho thấy với bộ dữ liệu diamonds, MLP không thực sự vượt trội.

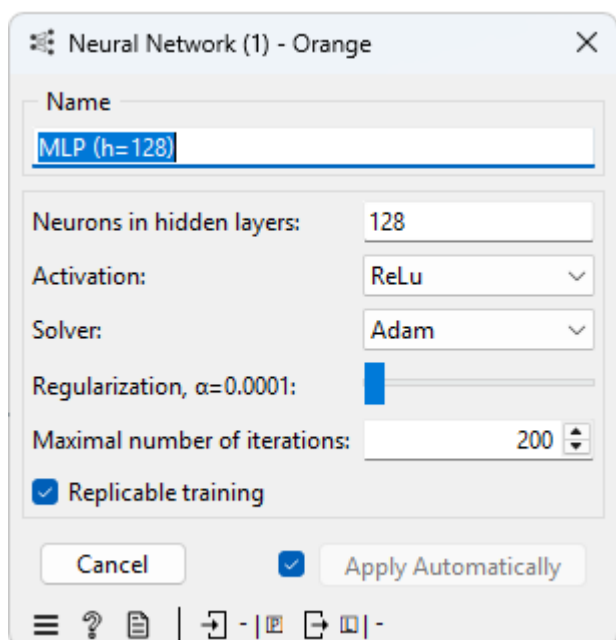
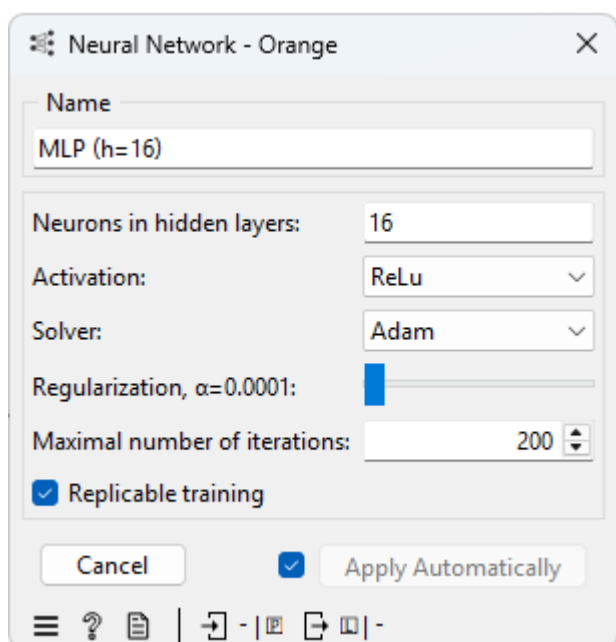
Bộ dữ liệu này có các đặc trưng đầu vào tương quan tuyến tính rất mạnh với nhãn giá trị (ví dụ: kích thước carat càng lớn, giá càng cao). Do đó, một mô hình tuyến tính cơ bản, dễ giải thích và tốn ít tài nguyên tính toán như Logistic Regression đã giải quyết bài toán một cách xuất sắc. Việc sử dụng MLP ở đây là "dùng dao mổ trâu giết gà", làm tăng độ phức tạp không cần thiết và dễ dẫn đến rủi ro overfitting.

Minh chứng

- 01 ảnh workflow Orange.



- 01 ảnh bảng kết quả hoặc cấu hình của widget Neural Network.



- Nếu có dùng baseline, chụp thêm 01 ảnh bảng Test & Score có ít nhất hai mô hình để đối chiếu

Test and Score - Orange

Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 66 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
MLP (h=16)	0.999	0.980	0.980	0.980	0.980	0.961
MLP (h=128)	0.998	0.980	0.980	0.980	0.980	0.960
Logistic Regression	0.998	0.979	0.979	0.979	0.979	0.958

Compare models by: Area under ROC curve

☐ Negligible diff.: 0.1

	MLP (h=16)	MLP (h=128)	Logistic Regression
MLP (h=16)		0.615	0.976
MLP (h=128)	0.385		0.951
Logistic Regression	0.024	0.049	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

- 3–5 dòng nhận xét về tác động của số nút ẩn.

Dựa trên kết quả Cross-validation, cả hai cấu hình MLP đều cho độ chính xác phân lớp (CA) rất cao và gần như tương đương nhau ở mức 0.980. Cấu hình thứ nhất có nhỉnh hơn một chút xíu về chỉ số AUC (0.999 so với 0.998) nhưng sự chênh lệch là không đáng kể (Negligible). Điều này khẳng định lại rằng bộ dữ liệu diamonds không quá phức tạp để phân tách. Việc tăng số lượng nút ẩn hay tăng độ phức tạp của mạng Nơ-ron trong trường hợp này không mang lại sự cải thiện thực sự nào, mà chỉ làm tốn thêm thời gian tính toán.

✓ PHẦN 3. GOOGLE COLAB VỚI THƯ VIỆN PYTHON

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

# Thiết lập MSSV làm hạt giống (seed) để đảm bảo tính tái lập
```

```
MSSV = 2212453
np.random.seed(MSSV)
tf.random.set_seed(MSSV)
```

✓ BƯỚC 1: Chuẩn bị bộ dữ liệu nhị phân và chia tập dữ liệu

```
# Tải bộ dữ liệu diamonds
df = sns.load_dataset('diamonds')

# Tạo nhãn phân lớp (g=3): 1 nếu thuộc nhóm premium (giá > trung vị), ngược lại là 0
median_price = df['price'].median()
df['is_premium'] = (df['price'] > median_price).astype(int)

# Tiền xử lý: Mã hóa các biến phân loại (One-Hot Encoding) và bỏ cột 'price' gốc
X = df.drop(columns=['price', 'is_premium'])
X = pd.get_dummies(X, drop_first=True)
y = df['is_premium']

# Chia train (60%), validation (20%), test (20%) với hạt giống từ MSSV
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=MSSV)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=MSSV)

# Chuẩn hóa dữ liệu (rất quan trọng cho MLP)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print(f"Kích thước tập Train: {X_train_scaled.shape}")
print(f"Kích thước tập Val: {X_val_scaled.shape}")
print(f"Kích thước tập Test: {X_test_scaled.shape}")
```

```
Kích thước tập Train: (32364, 23)
Kích thước tập Val: (10788, 23)
Kích thước tập Test: (10788, 23)
```

✓ BƯỚC 2 & 3: Huấn luyện MLP với h=16, eta=0.002 và biện pháp chống Overfitting

```
h = 16
eta = 0.002

# Xây dựng mô hình
model = Sequential([
    Dense(h, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2), # Biện pháp chống overfitting (Dropout)
    Dense(1, activation="sigmoid")
])

# Biên dịch mô hình với learning rate được chỉ định
optimizer = Adam(learning_rate=eta)
model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])

# Biện pháp chống overfitting (Early Stopping)
```



```

early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=1)

# Huấn luyện mô hình (lưu log huấn luyện vào biến history)
history = model.fit(
    X_train_scaled, y_train,
    epochs=100,
    batch_size=64,
    validation_data=(X_val_scaled, y_val),
    callbacks=[early_stopping],
    verbose=1 # Đảm bảo có log huấn luyện in ra màn hình
)

```

```

poch 1/100
usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
06/506 ----- 5s 5ms/step - accuracy: 0.9094 - loss: 0.2213 - val_accuracy
poch 2/100
06/506 ----- 5s 9ms/step - accuracy: 0.9687 - loss: 0.0825 - val_accuracy
poch 3/100
06/506 ----- 4s 6ms/step - accuracy: 0.9727 - loss: 0.0727 - val_accuracy
poch 4/100
06/506 ----- 3s 5ms/step - accuracy: 0.9742 - loss: 0.0677 - val_accuracy
poch 5/100
06/506 ----- 2s 3ms/step - accuracy: 0.9751 - loss: 0.0664 - val_accuracy
poch 6/100
06/506 ----- 1s 2ms/step - accuracy: 0.9759 - loss: 0.0625 - val_accuracy
poch 7/100
06/506 ----- 2s 3ms/step - accuracy: 0.9762 - loss: 0.0631 - val_accuracy
poch 8/100
06/506 ----- 2s 3ms/step - accuracy: 0.9758 - loss: 0.0601 - val_accuracy
poch 9/100
06/506 ----- 1s 2ms/step - accuracy: 0.9770 - loss: 0.0602 - val_accuracy
poch 10/100
06/506 ----- 1s 2ms/step - accuracy: 0.9770 - loss: 0.0591 - val_accuracy
poch 11/100
06/506 ----- 1s 2ms/step - accuracy: 0.9769 - loss: 0.0576 - val_accuracy
poch 12/100
06/506 ----- 1s 3ms/step - accuracy: 0.9776 - loss: 0.0572 - val_accuracy
poch 13/100
06/506 ----- 1s 2ms/step - accuracy: 0.9784 - loss: 0.0567 - val_accuracy
poch 14/100
06/506 ----- 1s 2ms/step - accuracy: 0.9781 - loss: 0.0550 - val_accuracy
poch 15/100
06/506 ----- 1s 2ms/step - accuracy: 0.9783 - loss: 0.0538 - val_accuracy
poch 16/100
06/506 ----- 2s 3ms/step - accuracy: 0.9781 - loss: 0.0537 - val_accuracy

```

✓ Vẽ learning curve

```

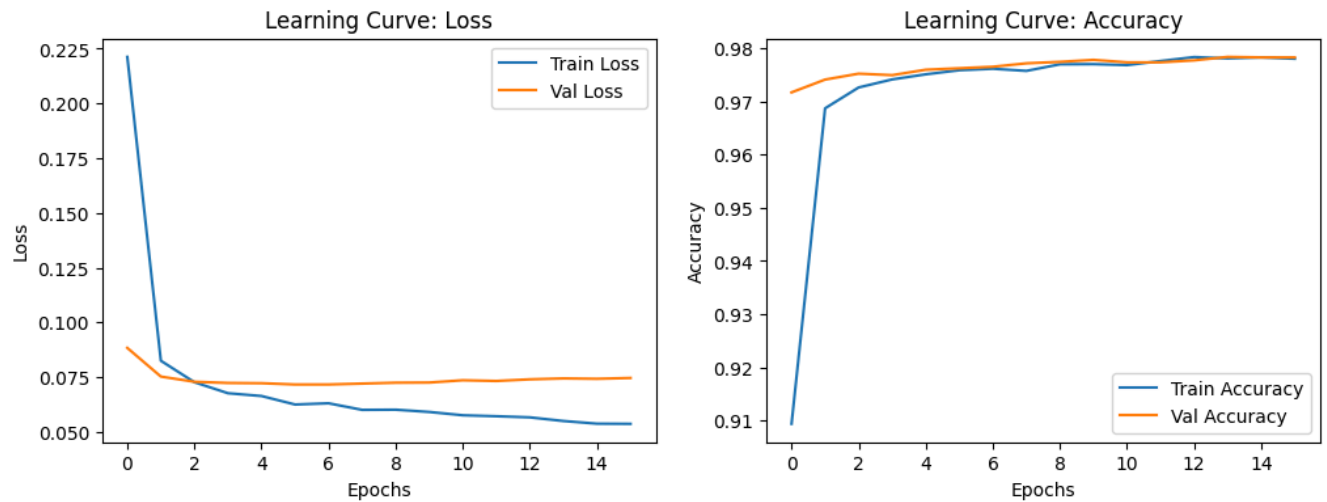
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Vẽ Loss
ax1.plot(history.history['loss'], label='Train Loss')
ax1.plot(history.history['val_loss'], label='Val Loss')
ax1.set_title('Learning Curve: Loss')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()

```

```
# Vẽ Accuracy
ax2.plot(history.history['accuracy'], label='Train Accuracy')
ax2.plot(history.history['val_accuracy'], label='Val Accuracy')
ax2.set_title('Learning Curve: Accuracy')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.show()
```



✓ BƯỚC 5: So sánh MLP với Logistic Regression (Baseline)

```
# Huấn luyện Logistic Regression
log_reg = LogisticRegression(random_state=MSSV)
log_reg.fit(X_train_scaled, y_train)

# Đánh giá trên tập test
y_pred_lr = log_reg.predict(X_test_scaled)
acc_lr = accuracy_score(y_test, y_pred_lr)

# Đánh giá MLP trên tập test
y_pred_mlp_prob = model.predict(X_test_scaled)
y_pred_mlp = (y_pred_mlp_prob > 0.5).astype(int).flatten()
acc_mlp = accuracy_score(y_test, y_pred_mlp)

print(f"Accuracy của Logistic Regression: {acc_lr:.4f}")
print(f"Accuracy của MLP: {acc_mlp:.4f}")
```

```
338/338 ————— 1s 2ms/step
Accuracy của Logistic Regression: 0.9808
Accuracy của MLP: 0.9800
```

✓ BƯỚC 6: In ra 5 dự đoán sai và phân tích

```
# Tìm các chỉ số dự đoán sai
incorrect_indices = np.where(y_pred_mlp != y_test.values)[0]
```

```
print(f"Tổng số dự đoán sai trên tập test: {len(incorrect_indices)} / {len(y_test)}")
print("Danh sách 5 dự đoán sai đầu tiên:\n")

for i in incorrect_indices[:5]:
    actual = y_test.iloc[i]
    predicted = y_pred_mlp[i]
    prob = y_pred_mlp_prob[i][0]

    # Lấy lại các đặc trưng gốc (chưa chuẩn hóa) để dễ đọc
    features = X_test.iloc[i]
    carat = features['carat']
    print(f"Mẫu index {i} | Carat: {carat} | Nhãn thật: {actual} | Dự đoán: {predicted}
```

Tổng số dự đoán sai trên tập test: 216 / 10788
Danh sách 5 dự đoán sai đầu tiên:

```
Mẫu index 55 | Carat: 0.78 | Nhãn thật: 0 | Dự đoán: 1 | Xác suất MLP: 0.7699
Mẫu index 124 | Carat: 0.7 | Nhãn thật: 0 | Dự đoán: 1 | Xác suất MLP: 0.5410
Mẫu index 159 | Carat: 0.87 | Nhãn thật: 0 | Dự đoán: 1 | Xác suất MLP: 0.5019
Mẫu index 200 | Carat: 0.7 | Nhãn thật: 1 | Dự đoán: 0 | Xác suất MLP: 0.3806
Mẫu index 333 | Carat: 0.75 | Nhãn thật: 1 | Dự đoán: 0 | Xác suất MLP: 0.4728
```

Việc chỉ nhìn vào Accuracy là chưa đủ vì Accuracy không phản ánh được sự mất cân bằng dữ liệu (nếu có), cũng như không cho biết mô hình tự tin đến mức nào ở các dự đoán sai (thể hiện qua xác suất). Đôi khi mô hình dự đoán sai nhưng xác suất xấp xỉ 0.5 (tức là mô hình phân vân), khác hoàn toàn với việc dự đoán sai nhưng mô hình lại cực kỳ tự tin (xác suất gần 1.0 hoặc 0.0).

Các yêu cầu phải trả lời

1. Dấu hiệu nào cho thấy mô hình bắt đầu overfit? Dấu hiệu overfitting thể hiện rõ nhất trên biểu đồ Learning Curve: Loss. Từ khoảng Epoch thứ 4 trở đi, đường Train Loss (màu xanh) tiếp tục có xu hướng giảm đều, trong khi đường Val Loss (màu cam) không giảm thêm mà bắt đầu đi ngang và có dấu hiệu nhích nhẹ lên. Sự phân kỳ (khoảng cách ngày càng rộng) giữa Train Loss và Val Loss chính là dấu hiệu cho thấy mô hình đang học thuộc lòng tập dữ liệu huấn luyện thay vì tổng quát hóa trên dữ liệu mới.
2. Biện pháp nào trong các biện pháp bạn thử có ích nhất? Biện pháp có ích nhất là Early Stopping (Dừng sớm). Nhìn vào trục hoành của biểu đồ, mô hình đã dừng lại ở Epoch 15 mặc dù được thiết lập huấn luyện tới 100 Epochs. Early Stopping đã theo dõi val_loss, phát hiện ra rằng sau khoảng Epoch 5, loss trên tập validation không còn được cải thiện (thậm chí tăng nhẹ). Nó đã chủ động dừng quá trình huấn luyện sớm và khôi phục lại bộ trọng số tốt nhất (restore_best_weights=True) để ngăn chặn việc mô hình bị overfit sâu hơn. Dropout cũng hỗ trợ việc làm mịn quá trình học, nhưng Early Stopping là biện pháp trực tiếp cắt đứt sự overfit.
3. Trong bối cảnh dữ liệu của bạn, MLP có thực sự đáng dùng hơn mô hình đơn giản không?

Trong bối cảnh bài toán phân loại giá kim cương này, MLP không thực sự đáng dùng hơn mô hình đơn giản như Logistic Regression.

Lý do:

- Dựa trên kết quả in ra, mô hình đơn giản (Logistic Regression) đạt độ chính xác (Accuracy) là 0.9808, nhỉnh hơn một chút so với mô hình phức tạp MLP là 0.9800.
- Dữ liệu kim cương (diamonds) có tính tuyến tính khá cao giữa các đặc trưng (ví dụ: kích thước carat càng lớn thì giá càng cao). Do đó, một mô hình tuyến tính đơn giản đã đủ sức phân tách các lớp một cách xuất sắc (đạt độ chính xác > 98%).
- Việc dùng MLP đòi hỏi tinh chỉnh nhiều siêu tham số (số nút ẩn, learning rate, epoch), tốn nhiều tài nguyên tính toán và dễ bị overfit hơn, nhưng lại không mang lại giá trị gia tăng nào về mặt độ chính xác so với mô hình cơ sở.

✓ Bài tập tự làm

- Tự quét ít nhất hai giá trị learning rate hoặc hai cấu hình số nút ẩn khác nhau rồi so sánh learning curve.
- Nhận xét cấu hình nào hội tụ ổn định hơn và cấu hình nào dễ overfit hơn.

```
import matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Định nghĩa 2 cấu hình số nút ẩn để so sánh
hidden_nodes_configs = [16, 128]
histories = {}

print("--- Bắt đầu huấn luyện để so sánh ---")
for nodes in hidden_nodes_configs:
    print(f"Đang huấn luyện mô hình với h = {nodes}...")

    # Xây dựng mô hình (Tạm thời bỏ Dropout và Early Stopping để ép mô hình bộc lộ n
    model_compare = Sequential([
        Dense(nodes, activation="relu", input_shape=(X_train_scaled.shape[1],)),
        Dense(1, activation="sigmoid")
    ])

    # Cùng sử dụng learning rate = 0.002
    model_compare.compile(optimizer=Adam(learning_rate=0.002),
                          loss='binary_crossentropy',
                          metrics=['accuracy'])

    # Huấn luyện cố định 30 epochs
    history_compare = model_compare.fit(
        X_train_scaled, y_train,
        epochs=30,
        batch_size=64,
        validation_data=(X_val_scaled, y_val),
        verbose=0 # Tắt log chi tiết từng epoch cho gọn màn hình
    )

    histories[nodes] = history_compare

print("--- Hoàn tất huấn luyện. Đang vẽ biểu đồ... ---")

# Vẽ Learning Curve so sánh
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
```

```
colors = {16: 'blue', 128: 'red'}
```

```
for nodes in hidden_nodes_configs:
```

```
    hist = histories[nodes]
```

```
    # Biểu đồ Loss
```

```
    ax1.plot(hist.history['loss'], label=f'Train Loss (h={nodes})', color=colors[nodes])
```

```
    ax1.plot(hist.history['val_loss'], label=f'Val Loss (h={nodes})', color=colors[nodes])
```

```
    # Biểu đồ Accuracy
```

```
    ax2.plot(hist.history['accuracy'], label=f'Train Acc (h={nodes})', color=colors[nodes])
```

```
    ax2.plot(hist.history['val_accuracy'], label=f'Val Acc (h={nodes})', color=colors[nodes])
```

```
ax1.set_title('So sánh Learning Curve: Loss')
```

```
ax1.set_xlabel('Epochs')
```

```
ax1.set_ylabel('Loss')
```

```
ax1.legend()
```

```
ax2.set_title('So sánh Learning Curve: Accuracy')
```

```
ax2.set_xlabel('Epochs')
```

```
ax2.set_ylabel('Accuracy')
```

```
ax2.legend()
```

```
--- Bắt đầu huấn luyện để so sánh ---
```

```
Đang huấn luyện mô hình với h = 16...
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:   
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Đang huấn luyện mô hình với h = 128...
```

```
--- Hoàn tất huấn luyện. Đang vẽ biểu đồ... ---
```

```
<matplotlib.legend.Legend at 0x7eaa99ed550>
```

